hw5

February 18, 2025

1 Homework 5

We have learned about the basics of using machine learning and deep learning for many computer vision problems, including object classification, semantic segmentation, object detection, etc. In this assignment, we will be building a framework for object classification using PyTorch.

Topics you will be learning in this assignment: * Defining datasets in PyTorch; * Defining models in PyTorch; * Specifying the training procedure; * Training and evaluating a model; * Tuning hyper-parameters.

2 0. Setup

[ ]:
```
# import os

# if not os.path.exists("CS131_release"):
#     # Clone the repository if it doesn't already exist
#     !git clone https://github.com/StanfordVL/CS131_release.git

# %cd CS131_release/winter_2025/hw5_release/
```

[ ]:
```
# Install the necessary dependencies
# (restart your runtime session if prompted to, and then re-run this cell)
# !pip install -r requirements.txt
```

2.1 1. Object classification on CIFAR10 with a simple ConvNet.

As one of the most famous datasets in computer vision, CIFAR10 is an object classfication dataset that consists of 60000 color RGB images in 10 classes, with 6000 images per class. The images are all at a resolution of 32x32. In this section, we will be working out a framework that is able to tell us what object there is in a given image, using a simple Convolution Neural Network.

2.1.1 1.1 Data preparation.

The most important ingredient in a deep learning recipe is arguably data - what we feed into the model largely determines what we get out of it. In this part, let's prepare our data in a format that will be best useable in the rest of the framework.

For vision, there's a useful package called `torchvision` that defines data loaders for common datasets as well as various image transformation operations. Let's first load and normalize the

training and testing dataset using `torchvision`.

As a quick refresher question. Why do we want to split our data into training and testing sets?

You answer here: In this way we can test our model's performance on new, unseen data, making sure that the model is not overfitted to the training set.

[9]:
```
import torch
import torchvision
import torchvision.transforms as transforms
import numpy as np
import random

# set random seeds
torch.manual_seed(131)
np.random.seed(131)
random.seed(131)
```

From this step, you want to create two dataloaders `trainloader` and `testloader` from which we will query our data. You might want to familiarize yourself with PyTorch data structures for this. Specifically, `torch.utils.data.Dataset` and `torch.utils.data.DataLoader` might be helpful here. `torchvision` also provides convenient interfaces for some popular datasets including CIFAR10, so you may find `torchvision.datasets` helpful too.

When dealing with image data, oftentimes we need to do some preprocessing to convert the data to the format we need. In this problem, the main preprocessing we need to do is normalization. Specifically, let's normalize the image to have 0.5 mean and 0.5 standard deviation for each of the 3 channels. Feel free to add in other transformations you may find necessary. `torchvision.transforms` is a good point to reference.

Why do we want to normalize the images beforehand?

HINT: consider the fact that the network we developed will be deployed to a large number of images.

You answer here: Normalized images will give a more stable graident decent process, makes the training easier. It will also make sure the data distribution is consistent, reducing the OOD case in the whild data.

[10]:
```
trainloader = None
testloader = None
batch_size = 4

### YOUR CODE HERE
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=transform)
```

```
testset = torchvision.datasets.CIFAR10(root='./data', train=False,␣
 ↪download=True, transform=transform)

trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,␣
 ↪shuffle=True, num_workers=2)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,␣
 ↪shuffle=False, num_workers=2)
### END YOUR CODE

# these are the 10 classes we have in CIFAR10
classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')

# running this block will take a few minutes to download the dataset if you␣
 ↪haven't done so
```

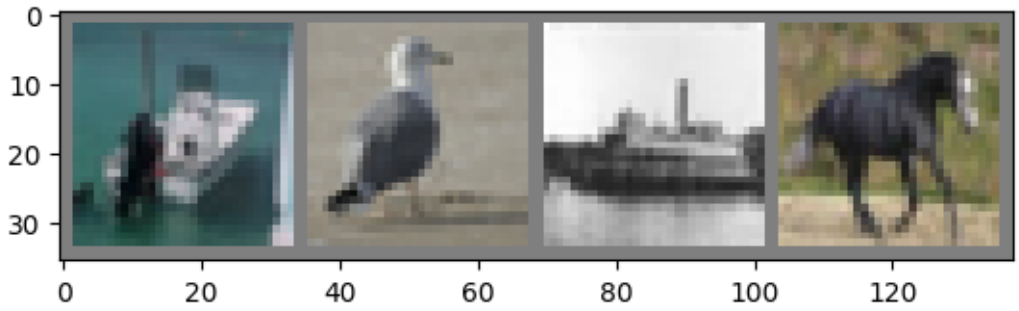
Let's plot out some training images to see what we are dealing with:

[4]:
```
import matplotlib.pyplot as plt

def imshow(img):
    img = img / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

# get some random training images
dataiter = iter(trainloader)
images, labels = next(dataiter)

# show images
imshow(torchvision.utils.make_grid(images))
# print labels
print(' '.join('%5s' % classes[labels[j]] for j in range(batch_size)))
```



```
 ship  bird  ship horse
```

2.1.2 1.2 Model definition.

Now we have the data ready, the next step is to define the model that we want to train on these data. Since CIFAR10 is a small dataset, we'll just build a very simple Convolutional Neural Network for our problem. The architecture of it will be (in order):

- 2D convolution: output feature channel number = 6, kernel size = 5x5, stride = 1, no padding;
- 2D max pooling: kernel size = 2x2, stride = 2;
- 2D convolution: output feature channel number = 16, kernel size = 5x5, stride = 1, no padding;
- 2D max pooling: kernel size = 2x2, stride = 2;
- Fully-connected layer: output feature channel number = 120;
- Fully-connected layer: output feature channel number = 84;
- Fully-connected layer: output feature channel number = 10 (number of classes).

Implement the `__init__()` and `forward()` functions in `Net`. As a good practice, `__init__()` generally defines the network architecture and `forward()` takes the runtime input `x` and passes through the network defined in `__init__()`, and returns the output.

[3]:
```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        ### YOUR CODE HERE
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
        ### END YOUR CODE

    def forward(self, x):
        ### YOUR CODE HERE
        x = self.pool1(F.relu(self.conv1(x)))
        x = self.pool2(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        ### END YOUR CODE
        return x
```

```
net = Net()
```

2.1.3 1.3 Loss and optimizer definition.

Okay we now have the model too! The next step is to train the model on the data we have prepared. But before that , we first need to define a loss function and an optimization procedure, which specifies how well our model does and how the training process is carried out, respectively. We'll be using Cross Entropy loss as our loss function and Stochastic Gradient Descent as our optimization algorithm. We will not cover them in detail here but you are welcome to read more on it. (this article and this article from CS231n would be a great point to start).

PyTorch implements very convenient interfaces for loss functions and optimizers, which we have put for you below.

[6]:
```
import torch.optim as optim
import torch.nn as nn

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

2.1.4 1.4 Kick start training.

What we have done so far prepares all the necessary pieces for actual training, and now let's kick start the training process! Running this training block should take just several minutes on your CPU.

[7]:
```
epoch_num = 2
for epoch in range(epoch_num):  # loop over the dataset multiple times

    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 2000 == 1999:    # print every 2000 mini-batches
            print('[%d, %5d] loss: %.3f' %
                  (epoch + 1, i + 1, running_loss / 2000))
```

```
            running_loss = 0.0

print('Finished Training')
```

```
[1,  2000] loss: 2.272
[1,  4000] loss: 1.903
[1,  6000] loss: 1.674
[1,  8000] loss: 1.588
[1, 10000] loss: 1.520
[1, 12000] loss: 1.462
[2,  2000] loss: 1.382
[2,  4000] loss: 1.359
[2,  6000] loss: 1.360
[2,  8000] loss: 1.327
[2, 10000] loss: 1.306
[2, 12000] loss: 1.269
Finished Training
```

The last step of training is to save the trained model locally to a checkpoint:

[8]:
```
PATH = './cifar_net.pth'
torch.save(net.state_dict(), PATH)
```

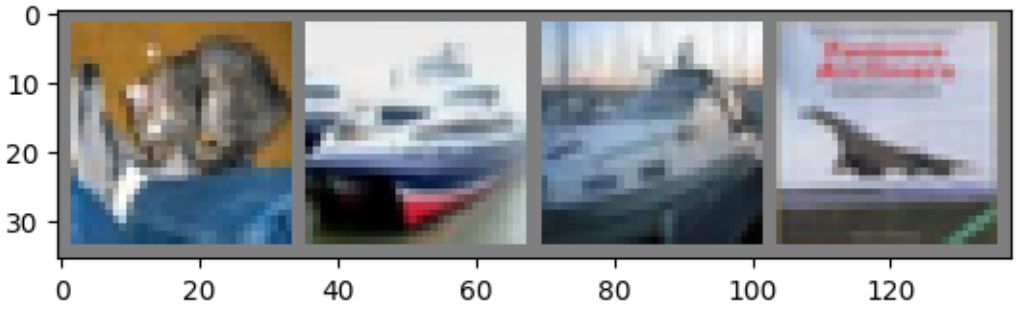
2.1.5 1.5 Test the trained model on the test data.

Remember earlier we split the data into training and testing set? Now we'll be using the testing split to see how our model performs on unseen data. We'll check this by predicting the class label that the neural network outputs, and comparing it against the ground-truth.

Let's first examine some data from the testing set:

[9]:
```
dataiter = iter(testloader)
images, labels = next(dataiter)

# print images
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join('%5s' % classes[labels[j]] for j in range(4)))
```



```
GroundTruth:    cat  ship  ship plane
```

Now, let's load in our saved model checkpoint and get its output:

[10]:
```
# load in model checkpoint
net = Net()
net.load_state_dict(torch.load(PATH))
```

[10]:
```
<All keys matched successfully>
```

[ ]:
```
# First, get the output from the model by passing in `images`;
# Next, think about what the model outputs mean / represent, and convert it to␣
 ↪the predicted class index (`predicted`);
# Finally, output the predicted class label (already done for you).

predicted = []
### YOUR CODE HERE
outputs = net(images)
_, predicted = torch.max(outputs, 1)
### END YOUR CODE
print('Predicted: ', ' '.join('%5s' % classes[predicted[j]]
                              for j in range(4)))
```

```
Predicted:    cat   car   car  ship
```

How does your prediction look like? Does that match your expectation? Write a few sentences to describe what you got and provide some analysis if you have any.

You answer here: It looks like pretty bad. None of the prediction is correct. However, it indeed makes reasonable guess in my mind. The boat looks pretty much like a car, and the plane is really like a ship.

Besides inspecting these several examples, let's also look at how the network performs on the entire testing set by calculating the percentage of correctly classified examples.

[13]:
```
correct = 0
total = 0
# since we're not training, we don't need to calculate the gradients for our␣
 ↪outputs
with torch.no_grad():
    for data in testloader:
        images, labels = data
        # Similar to the previous question, calculate model's output and the␣
 ↪percentage as correct / total
        ### YOUR CODE HERE
        outputs = net(images)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
```

```
        correct += (predicted == labels).sum().item()
        ### END YOUR CODE

print('Accuracy of the network on the 10000 test images: %d %%' % (
    100 * correct / total))
```

```
Accuracy of the network on the 10000 test images: 54 %
```

What accuracy did you get? Compared to random guessing, does your model perform significantly better?

You answer here: The Acc is 54%, compare to random guess (with expected acc 10%), it is much better.

Let's do some analysis to gain more insights of the results. One analysis we can carry out is the accuracy for each class, which can tell us what classes our model did well, and what classes our model did poorly.

[15]:
```
# prepare to count predictions for each class
correct_pred = {classname: 0 for classname in classes}
total_pred = {classname: 0 for classname in classes}

with torch.no_grad():
    for data in testloader:
        images, labels = data
        # repeat what you did previously, but now for each class
        ### YOUR CODE HERE
        outputs = net(images)
        _, predicted = torch.max(outputs, 1)
        for i in range(batch_size):
            label = labels[i]
            total_pred[classes[label]] += 1
            if predicted[i] == label:
                correct_pred[classes[label]] += 1
        ### END YOUR CODE

# print accuracy for each class
for classname, correct_count in correct_pred.items():
    accuracy = 100 * float(correct_count) / total_pred[classname]
    print("Accuracy for class {:5s} is: {:.1f} %".format(classname,
                                                        accuracy))
```

```
Accuracy for class plane is: 67.5 %
Accuracy for class car   is: 79.7 %
Accuracy for class bird  is: 13.6 %
Accuracy for class cat   is: 44.2 %
Accuracy for class deer  is: 47.6 %
Accuracy for class dog   is: 47.1 %
Accuracy for class frog  is: 68.0 %
Accuracy for class horse is: 60.7 %
```

```
Accuracy for class ship  is: 68.0 %
Accuracy for class truck is: 46.8 %
```

2.1.6 1.6 Hyper-parameter tuning.

An important phase in deep learning framework is hyper-parameter search. Hyper-parameters generally refer to those parameters that are **not** automatically optimized during the learning process, e.g., model architecture, optimizer, learning rate, batch size, training length, etc. Tuning these hyper-parameters could often lead to significant improvement of your model performance.

Your job in this section is to identify the hyper-parameters and tune them to improve the model performance as much as possible. You might want to refer to PyTorch documentation or other online resources to gain an understanding of what these hyper-parameters mean. Some of the options you might want to look into are: * Model architecture (number of layers, layer size, feature number, etc.); * Loss and optimizer (including loss function, regularization, learning rate, learning rate decay, etc.); * Training configuration (batch size, epoch number, etc.). These are by no means a complete list, but is supposed to give you an idea of the hyper-parameters. You are encouraged to identify and tune more.

Report in detail what you did in this section. Which of them improved model performance, and which did not?

You answer here: I set up an Optuna study to automatically search for the best hyperparameters. In our objective function, I sampled learning rate, momentum, batch size from predefined ranges. The model was then trained for a few epochs, and its performance on a validation set was measured to provide feedback to the optimizer.

[17]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
import torchvision.datasets as datasets
from torch.utils.data import DataLoader, random_split
import optuna
from tqdm import tqdm

def objective(trial):
    lr = trial.suggest_loguniform('lr', 1e-5, 1e-1)
    momentum = trial.suggest_float('momentum', 0.5, 0.99)
    batch_size = trial.suggest_categorical('batch_size', [4, 8, 16])

    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])
    dataset = datasets.CIFAR10(root='./data', train=True, download=True,␣
 ↪transform=transform)

    # Split dataset into training and validation sets
```

```
    train_size = int(0.8 * len(dataset))
    val_size = len(dataset) - train_size
    train_dataset, val_dataset = random_split(dataset, [train_size, val_size])

    train_loader = DataLoader(train_dataset, batch_size=batch_size,␣
 ↪shuffle=True, num_workers=2, pin_memory=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False,␣
 ↪num_workers=2, pin_memory=True)

    device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
    model = Net().to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(model.parameters(), lr=lr, momentum=momentum)

    epochs = 3
    for epoch in range(epochs):
        model.train()
        # Use tqdm to monitor training progress
        for images, labels in tqdm(train_loader, desc=f"Trial Epoch {epoch+1}/
 ↪{epochs}", leave=False):
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

    # Evaluate on the validation set
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = correct / total
    return accuracy

# Create a study object and optimize the objective function
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=10)

print("Best hyperparameters found:")
```

```
print(study.best_trial.params)
```

```
[I 2025-02-18 21:46:28,380] Trial 0 finished with value: 0.5483 and parameters:
{'lr': 0.0011457432142816608, 'momentum': 0.8894575267229191, 'batch_size': 8}.
Best is trial 0 with value: 0.5483.
[I 2025-02-18 21:47:38,970] Trial 1 finished with value: 0.5295 and parameters:
{'lr': 0.0028929211294720448, 'momentum': 0.8652313698249878, 'batch_size': 16}.
Best is trial 0 with value: 0.5483.
[I 2025-02-18 21:48:53,340] Trial 2 finished with value: 0.5684 and parameters:
{'lr': 0.00957645315526951, 'momentum': 0.5378543126080932, 'batch_size': 8}.
Best is trial 2 with value: 0.5684.
[I 2025-02-18 21:50:21,876] Trial 3 finished with value: 0.1005 and parameters:
{'lr': 0.09210580667115115, 'momentum': 0.9012144078310439, 'batch_size': 4}.
Best is trial 2 with value: 0.5684.
[I 2025-02-18 21:51:34,448] Trial 4 finished with value: 0.3235 and parameters:
{'lr': 0.0017348374084124112, 'momentum': 0.5046117378183083, 'batch_size': 16}.
Best is trial 2 with value: 0.5684.
[I 2025-02-18 21:52:53,878] Trial 5 finished with value: 0.4735 and parameters:
{'lr': 0.0030264016195168768, 'momentum': 0.5816960209713185, 'batch_size': 8}.
Best is trial 2 with value: 0.5684.
[I 2025-02-18 21:54:22,637] Trial 6 finished with value: 0.1579 and parameters:
{'lr': 7.274434363790377e-05, 'momentum': 0.6441356487692831, 'batch_size': 4}.
Best is trial 2 with value: 0.5684.
[I 2025-02-18 21:55:52,279] Trial 7 finished with value: 0.4261 and parameters:
{'lr': 0.01712343128547442, 'momentum': 0.7023117233935149, 'batch_size': 4}.
Best is trial 2 with value: 0.5684.
[I 2025-02-18 21:57:11,574] Trial 8 finished with value: 0.1012 and parameters:
{'lr': 1.86044973939339e-05, 'momentum': 0.7920994869381137, 'batch_size': 8}.
Best is trial 2 with value: 0.5684.
[I 2025-02-18 21:58:42,460] Trial 9 finished with value: 0.1801 and parameters:
{'lr': 0.034250775587598134, 'momentum': 0.6814108885253088, 'batch_size': 4}.
Best is trial 2 with value: 0.5684.

Best hyperparameters found:
{'lr': 0.00957645315526951, 'momentum': 0.5378543126080932, 'batch_size': 8}
```

2.2 2. Extra Credit: further improve your model performance

You have just tried tuning the hyper-parameters to improve your model performance. It's a very important part but not all! In this section, you are encouraged to read online to explore other options to further enhance your model. You may or may not need additional compute resources depending on what you do. But if you do need GPUs, Google Colab could be a great point to start.

Since this a free-form section, you should report here in detail what you have done, and feel free to submit any additional files if needed (e.g., additional code files). We'll be grading based on the effort you spend and the performance you achieved.

You answer here: In this section I set up another Optuna study to automatically search for the model architecture. In our objective function, we defined a dynamic CNN where key parameters

such as the number of convolutional layers, kernel sizes, initial number of filters, and batch size were sampled from predefined ranges. The rest of the approach stay the same with the previous section.

[14]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, random_split
import optuna
from tqdm import tqdm

# Define a CNN whose architecture is defined by hyperparameters from the trial
class DynamicCNN(nn.Module):
    def __init__(self, trial, num_classes=10):
        super(DynamicCNN, self).__init__()
        self.layers = nn.ModuleList()
        in_channels = 3

        # Choose the number of convolutional layers (e.g., between 2 and 4)
        n_conv_layers = trial.suggest_int("n_conv_layers", 2, 4)
        # Choose the initial number of filters
        init_filters = trial.suggest_categorical("init_filters", [16, 32, 64])
        out_channels = init_filters

        for i in range(n_conv_layers):
            # Suggest kernel size for each layer (odd values only: 3 or 5)
            kernel_size = trial.suggest_categorical(f"kernel_size_{i}", [3, 5])
            padding = kernel_size // 2  # to maintain spatial dimensions
            conv = nn.Conv2d(in_channels, out_channels,␣
↪kernel_size=kernel_size, stride=1, padding=padding)
            self.layers.append(conv)
            self.layers.append(nn.ReLU())
            # Use max pooling to reduce spatial dimensions by a factor of 2
            self.layers.append(nn.MaxPool2d(2, 2))
            in_channels = out_channels
            # Optionally double the number of channels for the next layer
            out_channels *= trial.suggest_categorical(f"channel_mult_{i}", [1,␣
↪2])

        # CIFAR10 images are 32x32; each maxpool divides spatial dims by 2.
        final_size = 32 // (2 ** n_conv_layers)
        self.flatten_dim = in_channels * final_size * final_size

        self.fc = nn.Linear(self.flatten_dim, num_classes)
```

```
    def forward(self, x):
        for layer in self.layers:
            x = layer(x)
        x = torch.flatten(x, 1)
        x = self.fc(x)
        return x

# Objective function for Optuna
def objective(trial):
    # Define data transformations and load CIFAR10 dataset
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])
    dataset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=transform)

    # Split dataset into training and validation sets
    train_size = int(0.8 * len(dataset))
    val_size = len(dataset) - train_size
    train_dataset, val_dataset = random_split(dataset, [train_size, val_size])

    # Suggest a batch size
    batch_size = trial.suggest_categorical("batch_size", [32, 64, 128])
    train_loader = DataLoader(train_dataset, batch_size=batch_size,␣
 ↪shuffle=True, num_workers=2)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False,␣
 ↪num_workers=2)

    device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

    # Create model with dynamic architecture
    model = DynamicCNN(trial, num_classes=10).to(device)
    criterion = nn.CrossEntropyLoss()

    lr = trial.suggest_loguniform("lr", 1e-4, 1e-2)
    optimizer = optim.Adam(model.parameters(), lr=lr)

    epochs = 3
    for epoch in range(epochs):
        model.train()
        for images, labels in tqdm(train_loader, desc=f"Trial Epoch {epoch+1}/
 ↪{epochs}", leave=False):
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
```

```
            loss.backward()
            optimizer.step()

    # Validate the model
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = correct / total
    return accuracy

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=20)

# Output the best trial
print("Best trial:")
trial = study.best_trial
print("  Accuracy: {:.4f}".format(trial.value))
print("  Architecture and hyperparameters:")
for key, value in trial.params.items():
    print("    {}: {}".format(key, value))
```

```
[I 2025-02-18 22:24:26,129] Trial 0 finished with value: 0.6154 and parameters:
{'batch_size': 64, 'n_conv_layers': 4, 'init_filters': 16, 'kernel_size_0': 3,
'channel_mult_0': 2, 'kernel_size_1': 5, 'channel_mult_1': 1, 'kernel_size_2':
5, 'channel_mult_2': 2, 'kernel_size_3': 5, 'channel_mult_3': 1, 'lr':
0.003188948887706216}. Best is trial 0 with value: 0.6154.
[I 2025-02-18 22:26:02,532] Trial 1 finished with value: 0.0959 and parameters:
{'batch_size': 64, 'n_conv_layers': 4, 'init_filters': 16, 'kernel_size_0': 5,
'channel_mult_0': 2, 'kernel_size_1': 3, 'channel_mult_1': 1, 'kernel_size_2':
5, 'channel_mult_2': 1, 'kernel_size_3': 5, 'channel_mult_3': 1, 'lr':
0.009981997780225713}. Best is trial 0 with value: 0.6154.
[I 2025-02-18 22:27:40,735] Trial 2 finished with value: 0.6731 and parameters:
{'batch_size': 128, 'n_conv_layers': 3, 'init_filters': 32, 'kernel_size_0': 3,
'channel_mult_0': 1, 'kernel_size_1': 3, 'channel_mult_1': 2, 'kernel_size_2':
3, 'channel_mult_2': 1, 'lr': 0.002364590808430174}. Best is trial 2 with value:
0.6731.
[I 2025-02-18 22:29:54,473] Trial 3 finished with value: 0.4157 and parameters:
{'batch_size': 64, 'n_conv_layers': 3, 'init_filters': 32, 'kernel_size_0': 5,
'channel_mult_0': 1, 'kernel_size_1': 5, 'channel_mult_1': 1, 'kernel_size_2':
5, 'channel_mult_2': 2, 'lr': 0.007038404746275253}. Best is trial 2 with value:
```

0.6731.
[I 2025-02-18 22:31:16,750] Trial 4 finished with value: 0.64 and parameters:
{'batch_size': 32, 'n_conv_layers': 2, 'init_filters': 16, 'kernel_size_0': 5,
'channel_mult_0': 2, 'kernel_size_1': 3, 'channel_mult_1': 2, 'lr':
0.0007581106128067303}. Best is trial 2 with value: 0.6731.
[I 2025-02-18 22:33:14,712] Trial 5 finished with value: 0.494 and parameters:
{'batch_size': 32, 'n_conv_layers': 3, 'init_filters': 32, 'kernel_size_0': 5,
'channel_mult_0': 2, 'kernel_size_1': 3, 'channel_mult_1': 1, 'kernel_size_2':
3, 'channel_mult_2': 1, 'lr': 0.004731680177242873}. Best is trial 2 with value:
0.6731.
[I 2025-02-18 22:37:52,809] Trial 6 finished with value: 0.6823 and parameters:
{'batch_size': 32, 'n_conv_layers': 4, 'init_filters': 64, 'kernel_size_0': 5,
'channel_mult_0': 1, 'kernel_size_1': 5, 'channel_mult_1': 2, 'kernel_size_2':
5, 'channel_mult_2': 2, 'kernel_size_3': 5, 'channel_mult_3': 2, 'lr':
0.001043626043167613}. Best is trial 6 with value: 0.6823.
[I 2025-02-18 22:40:06,188] Trial 7 finished with value: 0.4724 and parameters:
{'batch_size': 32, 'n_conv_layers': 2, 'init_filters': 32, 'kernel_size_0': 3,
'channel_mult_0': 2, 'kernel_size_1': 5, 'channel_mult_1': 2, 'lr':
0.0074266473390491765}. Best is trial 6 with value: 0.6823.
[I 2025-02-18 22:43:59,730] Trial 8 finished with value: 0.0994 and parameters:
{'batch_size': 128, 'n_conv_layers': 3, 'init_filters': 64, 'kernel_size_0': 5,
'channel_mult_0': 2, 'kernel_size_1': 5, 'channel_mult_1': 1, 'kernel_size_2':
5, 'channel_mult_2': 2, 'lr': 0.009113811452641929}. Best is trial 6 with value:
0.6823.
[I 2025-02-18 22:47:50,857] Trial 9 finished with value: 0.6951 and parameters:
{'batch_size': 32, 'n_conv_layers': 2, 'init_filters': 64, 'kernel_size_0': 5,
'channel_mult_0': 2, 'kernel_size_1': 5, 'channel_mult_1': 2, 'lr':
0.00030277363021265116}. Best is trial 9 with value: 0.6951.
[I 2025-02-18 22:51:06,365] Trial 10 finished with value: 0.5909 and parameters:
{'batch_size': 32, 'n_conv_layers': 2, 'init_filters': 64, 'kernel_size_0': 3,
'channel_mult_0': 1, 'kernel_size_1': 5, 'channel_mult_1': 2, 'lr':
0.00010756772862299182}. Best is trial 9 with value: 0.6951.
[I 2025-02-18 22:55:20,333] Trial 11 finished with value: 0.7042 and parameters:
{'batch_size': 32, 'n_conv_layers': 4, 'init_filters': 64, 'kernel_size_0': 5,
'channel_mult_0': 1, 'kernel_size_1': 5, 'channel_mult_1': 2, 'kernel_size_2':
5, 'channel_mult_2': 2, 'kernel_size_3': 3, 'channel_mult_3': 2, 'lr':
0.0004646750069409137}. Best is trial 11 with value: 0.7042.
[I 2025-02-18 22:58:39,739] Trial 12 finished with value: 0.6583 and parameters:
{'batch_size': 32, 'n_conv_layers': 2, 'init_filters': 64, 'kernel_size_0': 5,
'channel_mult_0': 1, 'kernel_size_1': 5, 'channel_mult_1': 2, 'lr':
0.00023414529459871858}. Best is trial 11 with value: 0.7042.
[I 2025-02-18 23:02:43,363] Trial 13 finished with value: 0.6975 and parameters:
{'batch_size': 32, 'n_conv_layers': 4, 'init_filters': 64, 'kernel_size_0': 5,
'channel_mult_0': 1, 'kernel_size_1': 5, 'channel_mult_1': 2, 'kernel_size_2':
3, 'channel_mult_2': 2, 'kernel_size_3': 3, 'channel_mult_3': 2, 'lr':
0.00034599536934887585}. Best is trial 11 with value: 0.7042.
[I 2025-02-18 23:05:57,944] Trial 14 finished with value: 0.6302 and parameters:
{'batch_size': 128, 'n_conv_layers': 4, 'init_filters': 64, 'kernel_size_0': 5,

```
'channel_mult_0': 1, 'kernel_size_1': 5, 'channel_mult_1': 2, 'kernel_size_2':
3, 'channel_mult_2': 2, 'kernel_size_3': 3, 'channel_mult_3': 2, 'lr':
0.0005231075494880746}. Best is trial 11 with value: 0.7042.
[I 2025-02-18 23:11:49,802] Trial 15 finished with value: 0.6636 and parameters:
{'batch_size': 32, 'n_conv_layers': 4, 'init_filters': 64, 'kernel_size_0': 5,
'channel_mult_0': 1, 'kernel_size_1': 5, 'channel_mult_1': 2, 'kernel_size_2':
3, 'channel_mult_2': 2, 'kernel_size_3': 3, 'channel_mult_3': 2, 'lr':
0.0013672553951398724}. Best is trial 11 with value: 0.7042.
[I 2025-02-18 23:15:47,791] Trial 16 finished with value: 0.7019 and parameters:
{'batch_size': 32, 'n_conv_layers': 4, 'init_filters': 64, 'kernel_size_0': 5,
'channel_mult_0': 1, 'kernel_size_1': 5, 'channel_mult_1': 2, 'kernel_size_2':
3, 'channel_mult_2': 2, 'kernel_size_3': 3, 'channel_mult_3': 2, 'lr':
0.0002619083772331307}. Best is trial 11 with value: 0.7042.
[I 2025-02-18 23:19:46,420] Trial 17 finished with value: 0.6201 and parameters:
{'batch_size': 32, 'n_conv_layers': 4, 'init_filters': 64, 'kernel_size_0': 5,
'channel_mult_0': 1, 'kernel_size_1': 5, 'channel_mult_1': 2, 'kernel_size_2':
3, 'channel_mult_2': 2, 'kernel_size_3': 3, 'channel_mult_3': 2, 'lr':
0.00013364306183463655}. Best is trial 11 with value: 0.7042.
[I 2025-02-18 23:22:19,700] Trial 18 finished with value: 0.6015 and parameters:
{'batch_size': 64, 'n_conv_layers': 3, 'init_filters': 64, 'kernel_size_0': 3,
'channel_mult_0': 1, 'kernel_size_1': 3, 'channel_mult_1': 2, 'kernel_size_2':
5, 'channel_mult_2': 2, 'lr': 0.00019256856566803255}. Best is trial 11 with
value: 0.7042.
[I 2025-02-18 23:23:55,768] Trial 19 finished with value: 0.4809 and parameters:
{'batch_size': 128, 'n_conv_layers': 4, 'init_filters': 16, 'kernel_size_0': 5,
'channel_mult_0': 1, 'kernel_size_1': 5, 'channel_mult_1': 2, 'kernel_size_2':
5, 'channel_mult_2': 1, 'kernel_size_3': 3, 'channel_mult_3': 2, 'lr':
0.0005254050814738397}. Best is trial 11 with value: 0.7042.

Best trial:
  Accuracy: 0.7042
  Architecture and hyperparameters:
    batch_size: 32
    n_conv_layers: 4
    init_filters: 64
    kernel_size_0: 5
    channel_mult_0: 1
    kernel_size_1: 5
    channel_mult_1: 2
    kernel_size_2: 5
    channel_mult_2: 2
    kernel_size_3: 3
    channel_mult_3: 2
    lr: 0.0004646750069409137
```